# Introduction to Image Analysis

# Compulsory Assignment 2

Allan Rasmusson

2026.10.21, *Version 1.0*

# Assignment Description

This document (7 pages in total) describes the second compulsory assignment for the course "Introduction to Image Analysis" taking place at Faculty of Mathematics and Informatics, Vilnius University, Lithuania, Autumn 2026. This is the second of a total of three assignments and is scored between 0 and 100 points and weighted by 30% in the overall grade for the course. **NB:** *all* assignments must be handed in to qualify for reexamination, but you are allowed to hand in a single page for a score of 0.

The assignment concerns the practical exercises in weeks 8 to 12 of the course, and, can be understood as giving a summary of the material covered in the course so far, a practical implementation of it with some specific tasks to solve using it. You may assume that any topics from the first assignment has been explained, but implementations must build upon the implementations from the previous assignment so any relavant missing parts must be implemented.

The student must include a written report with explanations of the background material, explained for an audience assumed not already familiar with it. Additionally, the implementation must be explained, including any choices and handling of special cases. The student may (you are *encouraged* to) reorder the topics, exercises and descriptions of implementations to optimize the understanding of the material as a whole. This includes considering which illustrations (both diagrams and outputs from the implementation) to include to best support explanations and proofs-of-concept. In other words: the report must in itself contain the necessary illustrations to argue that your explanations/example are correct *without* relying on readers running the code.

Both the report and the implemented programs (C++ or Python) must be handed in. You must include a description of how each example can be run on a command line in Ubuntu Linux. For the C++ code you must also describe how to compile it using a combination of `make` and `qmake` as well.

It is perfectly fine to hand in multiple pieces for code different examples or solutions to the exercises, and it's also acceptable that the examples must be compiled separately using `qmake` and/or `make`. Examples needing alterations (e.g. commenting certain sections in/out) of the handed-in code should not be expected to be run and may thus not contribute to the overall score.

Although the assignment covers the practical exercises already handed out, some clarification and specific tasks are described in the following sections. Some specific data is included with this assignments (`ImgSetCa2.zip`), but feel free to support you explanations using any other data generated by yourself or handed out during the course.

The report and code will contribute equally to the final score and must be handed in as a single zip-file uploaded in the VU Emokymai system. Deadline is *Sunday 2026.11.29*, at 23.59 (1 minute to *midnight*).

## Code Limitations

### C++

The code builds on the implementations from Compulsory Assignment 1.

You are allowed to use basic standard libraries like `iostream`, `string`, `cstdint`, `limits` and `cmath` for rounding. `vector` is allowed for small dynamic lists, but *not* for image pixel data.

For linear algebra, use the Eigen library by including `Eigen/Core` and `Eigen/LU`, and for the Fourier Transform use `fftw3` library.

Compile the image processing implementations as a stand-alone library, which is then used in (linked with) one or multiple example files. Use the handed out image viewer based on Qt for image viewing. You may use the compilation process in the code hand-out which is described in the slides from Week 2.

### Python

Python implementations also build upon implementations from assignment 1, so you are only allowed to use the following libraries:

```
from libtiff import TIFF
from matplotlib import pyplot as plt
import numpy as np
import math
import sys
```

with `numpy.matmul` and `numpy.linalg` used for matrix/vector calculations. For the Fourier transform, you must use `numpy.fft`.

Image viewing is restricted to 8bit images as follows:

```
# Grayscale
plt.imshow(aGrayScaleImage, cmap='gray', interpolation='nearest', vmin=0, vmax=255)
# RGB
plt.imshow(someColorImage, interpolation='nearest',vmin=0, vmax=255)
```

**Some Advice:**

- Read the assignment description *at least* twice!
- Ask questions as early as possible.
- If you are having problems with some parts of the exercises, continue to the next and make it work with what you've got.
- Make sure you have access to the handed out files.
- Think about which part of the material is relevant for each weekly topic and include this in the report.
- Think about which examples and illustrations you need to support your hypotheses/explanations.

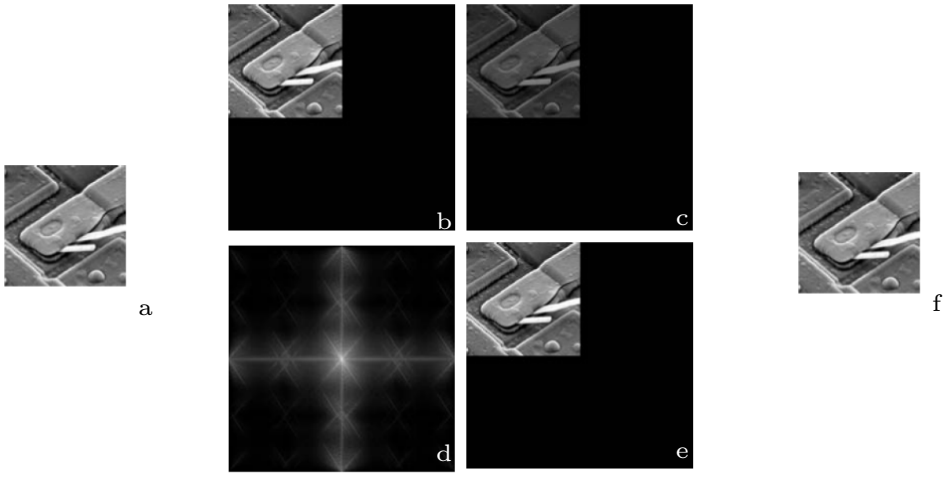
# 1 Assignment Details

## 1.1 Fourier Transform

**Purpose:** Illustrate sufficient understanding of the Fourier transform to implement the necessary steps for performing a Fourier Transform given an implementation of the Discrete Fourier Transform algorithm. C++ implementations will use the `fftw3` library and Python implementations will use the `numpy.fft` as explained in the exercises for weeks 8–12.

**Implementations and Tasks:**

- Implement the following steps needed to perform the forwards and backwards Fourier transforms of a single image without any additional alterations:
  - a: input image $M \times N$.
  - b: padded image $2M \times 2N$.
  - c: b shifted for periodicity.
  - d: **DFT**, $F(u, v)$ of c.
    - Filtering (by $F(u, v)H(u, v)$) will be performed here.
    - For now, just do visualization of $F$.
  - e: **IDFT** of d shifted for periodicity.
  - f: after image is upper left quadrant of e.
  - NB: Some of the steps will have to pay attention to **DFT** scaling factor.
  - Without filter application ($H(u, v) = I$) steps e and f will show if the **DFT** and **IDFT** are applied correctly.




**Visualization:** Argue that the steps have been correctly implemented.

**Data:** Images handed out during the course.

## 1.2 Fourier Transform of generated images

**Purpose:** Building some intuition for what the result of the Fourier Transform represents using generated images.

**Implementations and Tasks:**

Explain the basic theory for different applications and properties of the Fourier Transform and what it leads you to expect when applying this to images with various content. Generate images with this content, apply the Fourier spectrum, and discuss if the results fit with your expectations.

To generate images of repeated patterns in image space you *may* take *inspiration* in the following (pseudo) code:

```
width = 256;
height = 256;
MyImage im = allocate new  MyImage of size widt x height;

float alphaX = 2.0f*M_PI / width
float alphaY = 2.0f*M_PI / height

for( yIter = 0; yIter < height; yIter++){
  for( xIter = 0; xIter < width; xIter++){
    float i = cos( ``some combination of alphaX|Y and x|yIter'')
    MyImage.setPixel(xIter,yIter, i);
  }
}
```

NB: This is *not* a complete example: you must consider choice of format for `MyImage` and number of intensity levels, affected pixel rows/columns etc.

**About examples:** Generate a few examples that show the properties of the Fourier transform. Explain your choices for the generated images, *argue* using the theory for what you expect the resulting Fourier spectrum of the generated images should be, and check if the outcome of transforming them meets your expectations.

## 1.3 Filtering in Frequency Space

**Purpose:** Understanding/explaining how filtering works in frequency space as opposed to filtering in image space and some of the benefits/drawbacks of filtering in each space.

**Implementations and Tasks:**

- I: Implement high-pass and low-pass versions of the *Ideal*, *Butterworth* and *Gaussian* frequency space filters and give examples of each filter.
  Extend the implementation in 1.1 to apply the filtering.
- II: Explain how operations can be performed in both image and frequency space and show benefits/drawbacks of:
  - Blurring with an averaging filter in image space vs. corresponding frequency space operation.
  - Blurring with a Gaussian filter in image space vs. corresponding frequency space operation.
  - Differentiation in image space vs. corresponding frequency space operation.
  - Band filtering in frequency space vs. corresponding image space operation.
  - Illustrate (and explain) why filtering by ideal frequency filters leads to ringing artifacts.

  *Hint:* It may be useful to know that any linear filter $w$ in image space can be padded with 0's without changing the results.

**Visualizations:** Any number you consider relevant for the tasks above. You may use any image from the weekly exercises and this assignment, or generate images.

## 1.4 Image Restoration - Noise Removal

**Purpose:** Gain practical experience with:

- The image degradation/restoration model.
- Basic noise identification.
- Noise removal in image space.
- Noise removal in frequency space.

**Implementations**

- Filtering in image space (reuse from Compulsory Assignment 1).
- Adaptive filters.
- Band-pass and reject filters.
- Notch filters based on frequency filters from Section 1.3.
- Error measures (also called Objective fidelity criteria).

**Tasks:**

- I: Periodic noise
  In the data accompanying this assignment is the image `Kidney1-Crop.tif` (original), and two noisy versions of it: `Kidney1-Crop-Noise1.tif`, `Kidney1-Crop-Noise2.tif`
  - Remove the periodic noise from `Kidney1-Crop-Noise1.tif` using band filters and notch filters. Discuss if band or notch is best or if it doesn't matter.
  - Use the same approach to try and remove periodic noise from `Kidney1-Crop-Noise2.tif`. Explain why or why not this approach is successful.
- II: In the data accompanying this assignment is the image `Thorax.tif` (original), and two noisy versions of it: `Thorax-Noise1.tif`, `Thorax-Noise2.tif`
  - Remove noise as you believe is best possible with the methods covered. Describe the steps you take, and the choices you make and how you evaluate your results.

**Data:** Images included with Compulsory Assignment 2.